

Keywords: efficient inference • KV-cache compression • process reward model • chain-of-thought reasoning

Fewer Tokens, Smaller Cache: Reward-Coordinated Efficient Reasoning

Process Reward Signals Jointly Guide Cache Eviction and Generation Length to Halve Inference Cost Without Sacrificing Accuracy

By Qiyuan Zhu, Dezhi Li, Pengyu Cheng, Tianle Chen, Jiacheng Wang, Ruijie Shen, Hao Gu, Sida Lin, Zirui Liu, Jiacheng Liu, Sirui Han

arXiv:2608.04771 • Preprint, August 2026

Large Reasoning Models generate long chain-of-thought traces that strain inference budgets in two entangled ways: the KV-cache grows with context length, and token generation itself consumes compute. Existing compression methods attack only the cache and apply uniform eviction policies across the entire trajectory. This paper shows that a process reward model (PRM) naturally tracks *where* compression is safe and *when* generation can stop, enabling simultaneous reduction of both costs under a single reward-coordinated framework called RCER.

AT A GLANCE

Problem: LRM inference is expensive in both KV-cache memory and generated tokens; uniform compression sacrifices accuracy.

Why it matters: Efficient LRM deployment is blocked by inference cost; a principled two-sided solution would substantially lower the barrier.

Method: Use a process reward model to guide KV-cache eviction (high-reward steps tolerate more deletion) and generation stopping (stop when the reward signals converge).

Result: Simultaneous reduction in cache size and token count, with accuracy preserved at compression rates that defeat uniform baselines.

Evidence: Ablations confirm that reward-guided eviction outperforms attention-score and recency heuristics at matched compression ratios.

The Big Picture

Large Reasoning Models (LRMs) are chain-of-thought systems trained to produce extended internal deliberation before outputting a final answer. The deliberation is effective — LRMs achieve state-of-the-art performance on mathematical, scientific, and code reasoning tasks — but it is expensive. A single difficult-problem response may

span thousands of tokens of intermediate reasoning, all of which must be stored in the KV-cache during generation and computed token by token.

Two inference costs accumulate jointly: the KV-cache grows linearly with the generated sequence, consuming memory bandwidth and limiting batch size; and the number of generated tokens determines total compute via floating-point operations. These costs are interdependent: a smaller cache means the model has less context, which causes it to regenerate lost information and produce more tokens than it would with a full cache.

This paper identifies the joint nature of the problem and proposes RCER (Reward-Coordinated Efficient Reasoning) as the first method to simultaneously minimize both quantities under a single process reward signal.

Why Existing Approaches Fall Short

Cache-only compression reduces KV-cache size by evicting past tokens based on heuristics: recency (evict oldest tokens), attention score (evict least-attended tokens), or importance scores derived from gradient norms. These approaches are unaware of the reasoning trajectory’s progress and evict uniformly, even at critical steps where context loss causes the model to misidentify its reasoning state.

Generation length reduction without cache-awareness typically operates through early-exit mechanisms that terminate generation when some threshold is reached. Without understanding the quality of the reasoning state — only its length — these mechanisms can terminate prematurely, before the model has reached a confident conclusion.

Neither class of method accounts for the *interaction*: compressing the cache causes more generation, and shortening generation can leave the cache unnecessarily large. RCER resolves this by tying both decisions to the same quality signal.

Core Mechanism

RCER augments standard LRM decoding with a process reward model ϕ that is evaluated at each reasoning step k to produce a scalar quality estimate $r_k = \phi(x, t_{\leq k}) \in [0, 1]$, where x is the input problem and $t_{\leq k}$ is the reasoning trace up to step k .

The PRM drives two decisions:

1. **Cache eviction:** When r_k is high (the model has committed to a correct sub-conclusion), the cached representations of older tokens are less necessary — the model’s trajectory is converging. RCER evicts an amount proportional to r_k from the oldest entries in the cache.

2. **Generation stopping:** When r_k exceeds a convergence threshold τ and has been stable for δ steps, RCER terminates generation and commits the current answer, avoiding wasteful continuation of a completed reasoning chain.

No additional model beyond the PRM (which may be a lightweight verifier) is required.

Technical Formulation

Let π_θ be the LRM and $r_k = \phi(x, t_{\leq k})$ the PRM score at step k .

Cache eviction budget at step k :

$$b_k = \beta \cdot r_k \cdot |\mathcal{C}_k|$$

where $|\mathcal{C}_k|$ is the current cache size and β is a compression rate hyperparameter. Tokens are evicted from the oldest b_k positions.

Generation stopping criterion:

$$\text{stop if } r_k \geq \tau \quad \text{and} \quad \frac{1}{\delta} \sum_{j=k-\delta}^k r_j \geq \tau - \varepsilon$$

This ensures both that the current state is high-quality and that the quality has been stable (not just a local spike).

Effective inference cost:

$$\mathcal{L} = \underbrace{\sum_{k=1}^K |\mathcal{C}_k|}_{\text{cache cost}} + \lambda \cdot \underbrace{K}_{\text{token cost}}$$

RCER minimizes $\mathcal{L}$ under the constraint that accuracy remains within ε of unconstrained decoding.

Learning or Inference Procedure

RCER is a *training-free* inference-time method. The LRM π_θ and PRM ϕ are both used off-the-shelf without any fine-tuning. The only hyperparameters are:

- $\beta \in [0, 1]$: eviction rate (ablated: 0.1, 0.3, 0.5)
- $\tau \in [0, 1]$: convergence threshold (ablated: 0.7, 0.8, 0.9)
- δ : stability window in steps (ablated: 5, 10, 20)
- λ : weight trading off cache vs. token cost

What the Guarantee Says

No formal guarantee is provided. The empirical claim is that RCER simultaneously reduces both KV-cache occupancy and generated tokens while preserving accuracy within a specified tolerance of unconstrained decoding. The main theoretical insight — that PRM score correlates

with the reasoning state’s tolerance to context loss — is supported by ablation experiments rather than formal analysis.

Experimental Findings

- **Cache reduction:** RCER reduces peak KV-cache size by 40–55% on GSM8K, MATH-500, and LiveCodeBench at accuracy within 1% of unconstrained decoding.
- **Token reduction:** RCER simultaneously reduces generated tokens by 20–35% through early stopping at convergence.
- **Uniform baselines fail:** At 40% cache compression, attention-score eviction drops accuracy by 4–8%; RCER drops by less than 1%.
- **Interaction confirmed:** Experiments show that cache-only compression at rate β increases token count by 10–25%, confirming the interaction and the necessity of joint coordination.

Ablations and Interpretation

Eviction strategy: Reward-guided eviction consistently outperforms recency, attention-score, and random eviction at matched budgets. The advantage grows with compression ratio, indicating that the reward signal is most critical at high compression where wrong eviction choices compound.

PRM quality: A lightweight 1B-parameter verifier achieves 85–90% of the benefit of a full-scale PRM, suggesting the approach is practical even without access to a high-accuracy reward model.

Stopping ablation: Generation stopping without cache coordination saves tokens but wastes cache — the combined approach is strictly better than either alone on the joint cost metric.

Reference

Qiyuan Zhu, Dezhi Li, Pengyu Cheng, Tianle Chen, Jiacheng Wang, Ruijie Shen, Hao Gu, Sida Lin, Zirui Liu, Jiacheng Liu, Sirui Han. **Fewer Tokens, Smaller Cache: Reward-Coordinated Efficient Reasoning.** arXiv:2608.04771, August 2026. <https://arxiv.org/abs/2608.04771>

Keywords: multimodal pretraining • knowledge transfer • modality synergy • vision laziness

Towards Physics of Multimodal Pretraining: Knowledge Flow, Modality Synergy, Early Unification, and Recipes

Systematic Empirical Study Reveals Asymmetric Knowledge Flow, Data-Driven Synergy, and a Vision Laziness Phenomenon That Together Explain Why Early Joint Training Dominates

By Junlin Han, Shengbang Tong, David Fan, Minghao Chen, Philip Torr, Filippos Kokkinos, Mike Lewis

arXiv:2608.05000 • Preprint, August 2026

Training a single model to understand and generate across modalities has become the dominant paradigm in multimodal AI, yet the mechanisms by which modalities interact during joint pretraining remain poorly understood. This paper presents a systematic empirical investigation — a “physics” of multimodal pretraining — identifying four mechanistic principles that explain why certain design choices succeed: asymmetric knowledge flow, data-complexity-driven synergy, early unification superiority, and compute-efficient recipes that deliver strong models at 5% of naive training cost.

AT A GLANCE

Problem: Multimodal pretraining design is largely empirical; the mechanisms behind successful choices are unknown.

Why it matters: Principled design could drastically reduce the compute required to train capable multimodal models.

Method: Controlled ablations over modality combinations, training order, and architecture choices; validated by training 13.5B MoE models on 2T tokens.

Result: Four mechanistic principles identified; efficient recipes achieve competitive quality at 5% of naive joint-training compute.

Evidence: Cross-modal transfer measured via held-out evaluation; vision laziness confirmed by representation analysis.

The Big Picture

Modern multimodal models are trained to process and generate across language, visual understanding (image

captioning, visual question answering), and visual generation (text-to-image synthesis). The standard practice is joint pretraining on a mixture of all modalities simultaneously. This works, but the design space is vast: which modalities to combine, in what order, with what architecture, and for how long on each task. Each choice is typically resolved through expensive ablations with no theoretical grounding.

This paper asks: can we derive principled design rules from the mechanisms of multimodal training? By systematically varying one factor at a time and measuring the impact on all modalities, the authors identify four mechanistic principles that collectively constitute a “physics” of multimodal pretraining.

Why Existing Approaches Fall Short

The most common baselines for multimodal model design are:

Naive joint training: Mix all modality data from the start in fixed ratios. Works, but expensive and provides no insight into which choices matter.

Sequential pretraining: Pretrain on language, then adapt to vision via fine-tuning (e.g., LLaVA-style). Efficient but caps multimodal integration quality because the language representations are already frozen when vision is introduced.

Late alignment: Train unimodal encoders separately, then align them via a projection layer. Limited by the quality of the alignment stage and the incompatibility of separately-optimized representation spaces.

None of these approaches is informed by an understanding of *how* modalities interact. The four principles below provide that understanding.

Core Mechanism

Principle 1 — Asymmetric Knowledge Flow. Language knowledge transfers readily into visual understanding (via shared text-image pairs), but visual generation benefits less directly from language pretraining. The asymmetry means language pretraining is a high-value starting point for understanding tasks but should not be allowed to dominate the training objective for generation tasks.

Principle 2 — Data Complexity Drives Synergy. On simple datasets (small vocabulary, low intra-class variability), training on multiple modalities simultaneously induces modality competition: one modality dominates gradient updates and the others underfit. On complex, diverse datasets, the modalities are synergistic: each modality provides complementary learning signal that improves performance on the others. Architecturally, synergy is promoted by shared attention and normalization layers

with modality-specific feed-forward networks.

Principle 3 — Early Unification and Vision Laziness. Unifying modalities from the start of training outperforms sequential or late alignment. The paper identifies the mechanism: *vision laziness*. When visual input is introduced after strong language representations have formed, the model has already developed robust language priors. It then relies on those priors rather than learning genuinely visual representations. The visual encoder receives gradient signals that push it to produce language-compatible features rather than visually grounded ones. Early unification prevents this by forcing visual and language representations to co-evolve from the first gradient step.

Principle 4 — Efficient Recipes. Combining the first three principles, the authors derive pretraining recipes that achieve state-of-the-art multimodal performance using only 5% of the compute budget of naive joint training.

Technical Formulation

Let $\mathcal{D} = \{(x_i^L, x_i^V, x_i^G)\}$ be a dataset of language, visual understanding, and visual generation examples. A multimodal model f_θ is trained with a joint loss:

$$\mathcal{L}(\theta) = \alpha_L \mathcal{L}_L(\theta) + \alpha_V \mathcal{L}_V(\theta) + \alpha_G \mathcal{L}_G(\theta)$$

where $\alpha_L, \alpha_V, \alpha_G$ are modality weights. Standard approaches set these weights by data size. The paper proposes setting them adaptively based on the complexity and synergy of the current data mixture.

The synergy coefficient between modalities i and j at training step t is:

$$S_{ij}(t) = \frac{\partial \mathcal{L}_i / \partial \theta \cdot \partial \mathcal{L}_j / \partial \theta}{\|\partial \mathcal{L}_i / \partial \theta\| \cdot \|\partial \mathcal{L}_j / \partial \theta\|}$$

Positive S_{ij} indicates aligned gradients (synergy); negative indicates competition. Monitoring S_{ij} during training reveals when modalities are cooperating and when they are in conflict.

Learning or Inference Procedure

The study trains a series of controlled ablation models, holding constant the total compute budget while varying: (1) which modalities are present, (2) the order in which modalities are introduced, (3) the architecture of shared vs. modality-specific layers, and (4) the data mixture complexity. The final efficient recipe is validated by training 4×13.5B-parameter MoE models on 2T tokens.

What the Guarantee Says

No formal guarantee is provided. The empirical claim is that the four principles reliably improve multimodal

model quality across the tested model sizes (up to 13.5B parameters) and data scales (up to 2T tokens). The cross-modal transfer measurements and vision laziness analysis provide mechanistic evidence for why the principles hold, but formal generalization bounds are absent.

Experimental Findings

- **Knowledge flow:** Pretraining on language improves visual understanding by up to 8 points (ImageNet accuracy) but improves visual generation by less than 2 FID points. Visual generation pretraining improves visual understanding by 3–5 points.
- **Synergy:** On complex datasets, joint training improves each modality by 3–12 points over single-modality training. On simple datasets, joint training hurts performance by 2–5 points.
- **Vision laziness:** Models with late visual integration have visual encoder representations that are 60–70% correlated with language encoder representations, vs. 30–40% for early-unified models — confirming that vision laziness reduces visual grounding.
- **Efficient recipes:** The derived recipe matches the quality of naive joint training at 5% compute, and exceeds sequential pretraining at all compute levels tested.

Ablations and Interpretation

Architecture: Fully shared architectures suffer modality competition even on complex data. Fully separate (modality-specific) architectures cannot share knowledge. The intermediate design — shared attention and normalization, modality-specific feed-forward — achieves the best trade-off in all settings.

Data mixing: The optimal mixture ratio is task-dependent, but the synergy coefficient S_{ij} predicts which mixtures will be beneficial in advance, enabling adaptive data scheduling without exhaustive grid search.

Training order: The vision laziness effect is most pronounced when vision is introduced after 30% of the total training budget has elapsed. Introducing vision at 10% or earlier eliminates the effect; full early unification (from step 0) is optimal.

Reference

Junlin Han, Shengbang Tong, David Fan, Minghao Chen, Philip Torr, Filippos Kokkinos, Mike Lewis. **Towards Physics of Multimodal Pretraining: Knowledge Flow, Modality Synergy, Early Unification, and Recipes.** arXiv:2608.05000, August 2026. <https://arxiv.org/abs/2608.05000>

Keywords: flow matching • efficient generative models • image synthesis • multidimensional conditioning

XYZFlow: Scaling Multidimensional Shortcut Flows for Efficient Generative Modeling

Adding Temporal History and Spatial Context to Flow Matching Straightens Trajectories, Enabling High-Quality Generation in Fewer Steps Without a Teacher Model

By Jinxiu Liu, Xuanming Liu, Kangfu Mei, Yandong Wen, Weiyang Liu

arXiv:2608.12276 • Preprint, August 2026 • CUHK; Westlake University; Johns Hopkins University

Diffusion models produce state-of-the-art images but require dozens of iterative sampling steps. The standard acceleration path — distilling a slow teacher into a few-step student — inherits the teacher’s quality ceiling and demands careful training. XYZFlow takes a different route: it redesigns flow matching itself by conditioning probability paths on denoising history (temporal dimension Y) and neighboring spatial patches (spatial dimension Z), in addition to the standard denoising time (dimension X). Richer conditioning straightens the flow trajectory, enabling high-quality generation in far fewer steps with no teacher model required.

AT A GLANCE

Problem: Flow/diffusion models require many sampling steps for high quality; distillation-based speedups depend on teacher quality and are hard to train.

Why it matters: Efficient generation is critical for interactive and high-throughput image synthesis applications.

Method: Extend flow matching to three conditioning dimensions: denoising time (X), denoising history (Y), and spatial patch context (Z).

Result: Higher quality per function evaluation than distilled few-step models; no teacher model required; quality improves monotonically with conditioning.

Evidence: FID and CLIP-score improvements over 1–4 step baselines across multiple image benchmarks.

The Big Picture

Flow matching learns a continuous-time transformation that maps noise $z \sim \mathcal{N}(0, I)$ to data $x \sim p_{\text{data}}$ via a probability path $(p_t)_{t \in [0,1]}$. Standard flow matching parameterizes this path along a single temporal dimension t

(the denoising time), yielding a velocity field $v_\theta(z, t)$ that must integrate to transform noise into data in a finite number of steps.

The core challenge is trajectory curvature: curved probability paths require many small integration steps for accurate traversal. Straight paths can be traversed in fewer steps with larger step sizes. Existing efficient methods reduce curvature after the fact via distillation, but this is computationally expensive and inherits the teacher model’s limitations.

XYZFlow introduces two additional conditioning dimensions that straighten the path *by design* during training: the denoising history (accumulated context from prior steps) and the spatial context (generated patches in neighboring regions). By conditioning on more of the world state at each step, the model’s uncertainty about the optimal next step decreases, and the induced trajectory becomes straighter.

Why Existing Approaches Fall Short

One/two-step distillation (Consistency Models, LCM, TurboFlow) trains a student model to match the teacher’s output in one or two steps. Quality is bounded by the teacher, and the distillation objective is sensitive to hyperparameters. The student also cannot generalize beyond the teacher’s mode coverage.

Adversarial diffusion distillation (ADD, DMD) adds a discriminator to the distillation loss, improving one-step quality but further complicating training and introducing GAN instabilities.

Autoregressive generation achieves efficiency through sequential patch prediction but sacrifices the parallelism of diffusion-based approaches and requires discrete tokenization.

XYZFlow achieves the efficiency gains of distilled models without a teacher, and the expressivity gains of autoregressive models without discrete tokenization.

Core Mechanism

XYZFlow’s velocity field is conditioned on three quantities at each step:

- **X — denoising time t :** Standard conditioning on current noise level, shared with all flow matching models.
- **Y — denoising history $h_t = (z_{t_1}, z_{t_2}, \dots)$:** A memory of the partial trajectory, conditioning on states at a set of prior denoising times $t_1 > t_2 > \dots > t$. This non-Markovian conditioning allows the model to exploit accumulated context about the direction of the trajectory.
- **Z — spatial context s_i :** During spatial dimension

generation (sequential patch prediction), conditioning on the set of already-generated patches $s_{<i}$ provides spatial consistency that reduces uncertainty about each new patch’s target state.

The velocity field is:

$$v_\theta(z_t, t, h_t, s_{<i}) \approx \nabla_{z_t} \log p_t(z_t \mid h_t, s_{<i})$$

with richer conditioning providing a more informative gradient direction at each step.

Technical Formulation

Standard flow matching minimizes the conditional flow matching loss:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, z_0, z_1} [\|v_\theta(z_t, t) - (z_1 - z_0)\|^2]$$

where $z_t = (1 - t)z_0 + tz_1$ is the linear interpolation path. XYZFlow extends this to the multidimensional conditioned loss:

$$\mathcal{L}_{\text{XYZ}}(\theta) = \mathbb{E}_{t, z_0, z_1, h_t, s_{<i}} [\|v_\theta(z_t, t, h_t, s_{<i}) - (z_1 - z_0)\|^2]$$

The key theoretical result is that the *optimal* velocity $v^*(z_t, t, h_t, s_{<i})$ is straighter (lower curvature) than $v^*(z_t, t)$ whenever the conditioning variables h_t and $s_{<i}$ are informative about the target z_1 . This follows from the variance decomposition:

$$\mathbb{E}[\|z_1 - z_0\|^2] \geq \mathbb{E}[\|z_1 - \mathbb{E}[z_1 \mid h_t, s_{<i}, z_t]\|^2]$$

The residual uncertainty under conditioning is lower, implying the conditioned target is closer to the true target at each step.

Learning or Inference Procedure

Training follows standard flow matching: sample pairs (z_0, z_1) , sample time t , compute z_t , retrieve history h_t from prior steps of the current trajectory, retrieve spatial context $s_{<i}$ from previously generated patches, and minimize $\mathcal{L}_{\text{XYZ}}$.

At inference, generation proceeds in two nested loops:

1. **Temporal loop:** Take N_T denoising steps ($N_T = 2-8$ for efficient generation, vs. 50–1000 for standard diffusion). Maintain history h_t across steps.
2. **Spatial loop:** At each denoising step, generate patches sequentially, conditioning each patch on the already-generated patches. N_S patches are generated per step.

What the Guarantee Says

The paper proves that, under perfect conditioning, the XYZFlow velocity field induces a straighter trajectory

than standard flow matching, requiring fewer integration steps for the same approximation error. The practical guarantee is: for a fixed number of function evaluations (NFE), XYZFlow achieves lower FID than unconditioned flow matching and distilled models at matched NFE.

Experimental Findings

- **Quality vs. NFE:** At NFE= 2, XYZFlow outperforms LCM and TurboFlow by 3–5 FID points on ImageNet-256 and COCO.
- **No teacher required:** XYZFlow trained from scratch matches or exceeds the quality of distilled models trained from a strong teacher.
- **Monotonic improvement:** Adding Y conditioning over X improves FID by 2–3 points; adding Z conditioning further improves by 1–2 points. $X < XY < XYZ$ holds consistently across NFE settings.
- **Scaling:** Quality scales gracefully with NFE; there is no saturation plateau seen in distilled models at 2–4 steps.

Ablations and Interpretation

History length: Conditioning on 2–3 prior states (vs. just $t - 1$) captures most of the benefit; conditioning on the full history provides marginal additional gain at higher memory cost.

Spatial patch order: Raster-scan order outperforms random order, confirming that spatial locality in the conditioning signal matters.

Architecture: Cross-attention over history states outperforms concatenation for integrating the Y and Z conditioning signals.

Reference

Jinxu Liu, Xuanming Liu, Kangfu Mei, Yandong Wen, Weiyang Liu. **XYZFlow: Scaling Multidimensional Shortcut Flows for Efficient Generative Modeling**. arXiv:2608.12276, August 2026. <https://arxiv.org/abs/2608.12276>

Keywords: representational geometry • classification • prototype theory • universal representations

Sparse Prototype Code Underlies Classification and Prediction Across Modalities

A Universal Geometry of Within-Class Variability, Aligned With Class Centroids and Competing Classes, Emerges Across Vision, Audio, and Language Models

By Yehonatan Avidan, Daniel D. Lee, Haim Sompolinsky

arXiv:2608.15632 • Preprint, August 2026

Why do deep networks trained on classification tasks learn representations that generalize so broadly? This paper identifies a universal representational geometry — the Sparse Prototype Code — that appears across state-of-the-art models in vision, audio, and language. The key discovery: within-class variability in representation space is not random. Its classifier-relevant component aligns strongly with the class prototype centroid and with the centroids of a sparse set of competing classes. This structure explains inter-class confusion patterns, predicts transferability, and provides a unified framework for understanding why classification training produces broadly useful representations.

AT A GLANCE

Problem: The geometric structure of learned representations is poorly understood, limiting principled model design and analysis.

Why it matters: A universal geometry across modalities would explain why classification training generalizes and enable principled comparison of representations.

Method: Spectral analysis of within-class covariance matrices in final representation spaces of vision, audio, and language models.

Result: Sparse Prototype Code geometry confirmed across all tested architectures and modalities; predicts confusion patterns and transferability.

Evidence: 33 pages, 13 figures, 14 tables spanning ViT, ResNet, CLIP, wav2vec, BERT, and GPT-class models.

The Big Picture

Deep learning models trained on classification tasks have proven surprisingly versatile: representations learned to classify ImageNet transfer to medical imaging, satellite imagery, and fine-grained recognition; representations

learned on language classification transfer to generation, translation, and reasoning. The mechanism behind this broad transferability is not well understood.

The dominant hypothesis is that classification forces networks to learn disentangled, semantically meaningful features. But this is vague: what geometric structure do these features have in representation space? This paper provides a precise characterization: the Sparse Prototype Code. It shows that this structure is not specific to one modality or architecture but is a universal consequence of training on classification tasks.

Why Existing Approaches Fall Short

Neural collapse theory (Papayan et al., 2020) characterizes the geometry of the final layer at the end of training: class means collapse to the vertices of a simplex equiangular tight frame (ETF). This is a useful characterization of inter-class geometry but does not address *within-class* variability — the spread of examples around each class mean. Neural collapse predicts that within-class variability vanishes at convergence, which is not the case for practical models trained on realistic data.

CKA and representation similarity methods compare representations between models but do not characterize the intrinsic geometry of a single model’s representation space.

Prototype theory in cognitive science posits that categories are organized around prototypical examples, but does not make quantitative predictions about the geometric structure of computational model representations.

The Sparse Prototype Code bridges these gaps by providing a quantitative, testable geometric characterization of within-class variability.

Core Mechanism

For a model f_θ and class c , let $\mu_c = \mathbb{E}_{x \sim p(x|c)}[f_\theta(x)]$ be the class centroid and $\Sigma_c = \mathbb{E}_{x \sim p(x|c)}[(f_\theta(x) - \mu_c)(f_\theta(x) - \mu_c)^\top]$ be the within-class covariance.

The paper performs spectral analysis of Σ_c , decomposing it as $\Sigma_c = U_c \Lambda_c U_c^\top$. The key finding is that the leading eigenvectors $u_1, u_2, \dots$ of Σ_c align strongly with class centroids:

1. $u_1 \approx \mu_c / \|\mu_c\|$ (the class prototype direction)
2. $u_2, \dots, u_k \approx \mu_{c_j} / \|\mu_{c_j}\|$ for a sparse set of competing classes $\{c_j\}_{j=1}^k$, with $k \ll C$ (where C is the total number of classes)

The eigenvalue spectrum is sparse: a small number of eigenvalues are large (aligned with prototype and competing-class directions) and the remaining eigenvalues are small (random variability). This sparsity is the defining feature of the Sparse Prototype Code.

Technical Formulation

Formally, define the *prototype alignment* of class c :

$$A_{\text{proto}}(c) = \frac{u_1^\top \mu_c}{\|\mu_c\|}$$

and the *competing-class alignment* of class c with class c' :

$$A_{\text{comp}}(c, c') = \max_{j \geq 2} \frac{u_j^\top \mu_{c'}}{\|\mu_{c'}\|}$$

The Sparse Prototype Code hypothesis states:

$$A_{\text{proto}}(c) \approx 1 \quad \text{for all } c$$

$$\sum_{c' \neq c} \mathbf{1}[A_{\text{comp}}(c, c') > \tau] = k(c) \ll C$$

where $k(c)$ is a sparse count of strongly competing classes.

The paper verifies both conditions across all tested models and modalities.

Learning or Inference Procedure

The analysis is purely empirical and requires no training. For each model:

1. Extract final-layer representations for all examples in the test set.
2. Compute class centroids $\{\mu_c\}$.
3. Compute within-class covariance matrices $\{\Sigma_c\}$.
4. Perform eigendecomposition of each Σ_c .
5. Measure alignment of leading eigenvectors with class centroids.

All steps are straightforward linear algebra; no optimization is involved.

What the Guarantee Says

The paper provides a theoretical derivation showing that the Sparse Prototype Code is the *optimal* within-class geometry for a linear classifier that minimizes cross-entropy loss while maintaining within-class coherence. That is, gradient descent on classification loss with weight decay provably induces the Sparse Prototype Code in the final layer in the limit of large data and a sufficiently expressive model. This is not a finite-sample guarantee but provides a theoretical grounding for the empirical observations.

Experimental Findings

- **Vision (ViT, ResNet, CLIP):** Prototype alignment $A_{\text{proto}} > 0.95$ for all classes in all models. Average competing-class count $k(c) = 3\text{--}8$ out of 1000 classes (ImageNet). Competing classes match human-annotated confusable categories.

- **Audio (wav2vec):** Same geometry emerges in the audio representation space for AudioSet classification. Competing classes are acoustically similar sounds (e.g., “bark” and “growl”).
- **Language (BERT, GPT-class):** Sparse Prototype Code confirmed in text classification tasks. Competing classes correspond to semantically adjacent categories (e.g., “positive” and “mildly positive” sentiment).
- **Transferability prediction:** Models with higher prototype alignment and lower competing-class count transfer better to new tasks (Pearson $r = 0.78$).

Ablations and Interpretation

Does the geometry require classification training?

Models trained with contrastive losses (CLIP, SimCLR) show partial but weaker Sparse Prototype Code structure, with higher competing-class count and lower prototype alignment. Supervised classification induces stronger structure.

Does architecture matter? The geometry is consistent across ViT and ResNet at matched model capacity, suggesting it is a property of the training objective rather than the architecture.

Depth: The Sparse Prototype Code is weakest in early layers and strongest in the final representation layer, consistent with hierarchical abstraction during training.

Reference

Yehonatan Avidan, Daniel D. Lee, Haim Sompolsky. **Sparse Prototype Code Underlies Classification and Prediction Across Modalities.** arXiv:2608.15632, August 2026. <https://arxiv.org/abs/2608.15632>

Keywords: vision-language-action models • credit assignment • robotic manipulation • GRPO

Temporal GRPO: Beyond Trajectory-Level Credit in Vision-Language-Action Reinforcement Learning

Decomposing Trajectory Rewards Into Step-Level Signals Improves Sample Efficiency and Task Success in Robotic Manipulation with VLA Policies

By Yao Zhou, Hang Gao, Fengge Wu, Changwen Zheng, Wenwen Qiang

arXiv:2608.13026 • Preprint, August 2026 • Robotics (cs.RO)

Reinforcement learning for Vision-Language-Action (VLA) models assigns a single trajectory-level reward to the entire action sequence, treating every step as equally responsible for success or failure. This coarse credit assignment conflates critical actions (precise contact) with inconsequential ones (stable approach), producing an uninformative gradient signal that slows learning. Temporal GRPO extends Group Relative Policy Optimization with a temporal credit decomposition mechanism, distributing trajectory rewards to individual steps based on their temporal context, and demonstrates improved sample efficiency and task success on robotic manipulation benchmarks.

AT A GLANCE

Problem: Trajectory-level RL credit assignment for VLA models fails to distinguish the contribution of individual actions to task outcomes.

Why it matters: VLA manipulation tasks have long action horizons where a few critical steps determine success; coarse credit wastes gradient budget.

Method: Decompose trajectory reward into step-level credits conditioned on temporal position and local state transitions within the GRPO framework.

Result: Improved sample efficiency and task success rate over standard GRPO on multi-step robotic manipulation benchmarks.

Evidence: Gains replicated across multiple manipulation tasks; ablations confirm temporal conditioning is the key driver.

The Big Picture

Vision-Language-Action (VLA) models are large multi-modal policies that take images of the scene and a language instruction as input and output a sequence of robot actions. They have achieved impressive results on diverse

manipulation tasks by combining the knowledge of large pretrained vision-language models with task-specific RL fine-tuning.

The RL fine-tuning phase typically uses algorithms like GRPO, which generates groups of G trajectories from the current policy, computes a binary reward (did the task succeed?), and uses the relative ranking within the group to compute policy gradient updates. Every action in a successful trajectory receives a positive update; every action in a failed trajectory receives a negative update.

This trajectory-level assignment is coarse. A grasping task with 80 action steps may succeed or fail based on a single step — the precise contact between the gripper and the object. Temporal GRPO identifies that step and concentrates gradient signal on it, leaving the uninformative steps with near-zero gradient.

Why Existing Approaches Fall Short

Standard GRPO assigns trajectory reward uniformly across all steps:

$$\nabla \mathcal{L} = \frac{1}{G} \sum_{g=1}^G \hat{A}_g \cdot \sum_{t=1}^T \nabla \log \pi_{\theta}(a_{g,t} | s_{g,t})$$

where $\hat{A}_g = R_g - \text{baseline}$ is the group-relative advantage. Every step t in trajectory g receives the same weight $\hat{A}_g$.

Dense reward shaping assigns step-level rewards via a hand-designed function (e.g., distance to goal, object height). This works for simple tasks but requires domain expertise and does not generalize across task families.

Hindsight reward assignment assigns credit based on outcome retrospectively but does not distinguish which actions were causally important for that outcome.

Temporal GRPO avoids hand-designed rewards and operates within the standard GRPO framework, requiring only a temporal credit decomposition function that is learned from the trajectory data.

Core Mechanism

Temporal GRPO decomposes the trajectory-level advantage $\hat{A}_g$ into step-level advantages $\hat{a}_{g,t}$ using a temporal credit function $c(t, T, s_{g,t})$ that depends on:

- **Temporal position t/T :** Early steps (approach) and late steps (contact, post-grasp) receive different baseline credit weights.
- **State transition:** The change in scene state $\Delta s_{g,t} = s_{g,t+1} - s_{g,t}$ identifies steps where significant state changes occurred (contact, placement). High-change steps receive higher credit.

The group-relative baseline is computed at the step level: for step t across all G trajectories, the baseline is the mean

step-level credit over the group:

$$\hat{a}_{g,t} = c(t, T, s_{g,t}) \cdot \hat{A}_g - \bar{c}_t$$

where $\bar{c}_t = \frac{1}{G} \sum_{g'=1}^G c(t, T, s_{g',t}) \cdot \hat{A}_{g'}$.

Technical Formulation

The Temporal GRPO policy gradient objective is:

$$\mathcal{L}_{\text{TG}}(\theta) = -\frac{1}{G} \sum_{g=1}^G \sum_{t=1}^T \hat{a}_{g,t} \cdot \log \pi_{\theta}(a_{g,t} \mid s_{g,t})$$

with credit function:

$$c(t, T, s_{g,t}) = w_{\text{pos}}(t/T) \cdot w_{\text{trans}}(\|\Delta s_{g,t}\|)$$

where w_{pos} is a learned position-dependent weight function (modeled as a shallow MLP) and w_{trans} is a monotone function of state transition magnitude.

The position weight w_{pos} is trained jointly with π_{θ} via a meta-learning objective that maximizes the correlation between step-level advantage and final task success probability:

$$\mathcal{L}_{\text{meta}}(w) = -\text{Corr}(\hat{a}_{g,t}, P(\text{success} \mid s_{g,1:t}))$$

where $P(\text{success} \mid s_{g,1:t})$ is estimated by a lightweight success predictor trained on the same trajectory data.

Learning or Inference Procedure

Training alternates between two updates:

1. **Policy update:** Sample G trajectories from π_{θ} , compute $\hat{a}_{g,t}$ using current w_{pos} , update θ via $\mathcal{L}_{\text{TG}}$.
2. **Credit function update:** Update w_{pos} via $\mathcal{L}_{\text{meta}}$ using the current trajectory data.

The success predictor is updated with every new trajectory batch. At inference, the VLA policy π_{θ} is used directly; no credit function is needed.

What the Guarantee Says

No formal guarantee is provided. The empirical claim is that temporal credit decomposition consistently improves sample efficiency (fewer trajectories to reach a target success rate) and asymptotic task success rate over standard GRPO, across the tested manipulation tasks. The meta-learning correlation objective provides a principled basis for learning the credit function, but convergence properties are empirical.

Experimental Findings

- **Sample efficiency:** Temporal GRPO reaches 80% task success in 40–55% fewer environment interactions

than standard GRPO across grasping, placing, and stacking tasks.

- **Asymptotic success:** Final success rates improve by 5–12 percentage points on complex long-horizon tasks (4+ subtasks).
- **Long-horizon advantage:** Improvement scales with task horizon. Short tasks (1–2 steps) show minimal gain; tasks with 6+ action phases show the largest improvements.
- **Contact step analysis:** The credit function concentrates >60% of the total trajectory credit on the 2–5 contact steps in a typical 80-step grasp trajectory, confirming that temporal credit is identifying the causally important steps.

Ablations and Interpretation

Position weighting only (removing state-transition weighting) captures 60–70% of the full benefit, confirming that temporal position is the dominant signal. The state-transition term adds the remainder.

Fixed vs. learned position weight: A hand-designed position weight (e.g., linear ramp emphasizing later steps) achieves 80% of the benefit of the learned weight on structured manipulation tasks, suggesting the timing pattern is partially predictable from task structure.

Group size G : Temporal GRPO achieves the same accuracy as standard GRPO with $G = 8$ using $G = 4$, halving the rollout cost for the same gradient quality.

Reference

Yao Zhou, Hang Gao, Fengge Wu, Changwen Zheng, Wenwen Qiang. **Temporal GRPO: Beyond Trajectory-Level Credit in Vision-Language-Action Reinforcement Learning**. arXiv:2608.13026, August 2026. <https://arxiv.org/abs/2608.13026>